

MiniANN Early Prototype: Mentor Meeting 1 Design Overview

Problem Statement 5: A Simple ANN Library in C++
Team of 4 — C++17, no external ML frameworks

October 1, 2026

Abstract

This report documents the **initial-phase prototype** of MiniANN: a from-scratch, object-oriented neural network library in modern C++17. At the Mentor Meeting 1 milestone the architecture and 4-member module division are frozen for validation, and exactly one functional module is complete: the **polymorphic activation subsystem** (`IActivation` with `Sigmoid`, `Tanh`, `ReLU`). A minimal forward-only skeleton (`Neuron` $\rightarrow$ `Layer` $\rightarrow$ `NeuralNetwork` $\rightarrow$ `predict`) proves the composition design compiles and runs, while training, dataset ingestion, and evaluation exist only as explicit `logic_error` stubs marking each member’s future work. We explain the purpose of this snapshot, why each design choice was made, what the code does module by module, how it was verified, and what is deliberately left for the full MVP (cf. `MiniANN_MVP/`).

Contents

1	Purpose of this prototype	2
1.1	Why clarity over speed	2
1.2	Relation to the full MVP	2
2	What the prototype does (and does not do)	2
3	How we did it: module by module	3
3.1	Common types	3
3.2	Activation subsystem (the completed piece)	3
3.3	Neuron: the smallest unit	3
3.4	Layer and network: composition over inheritance	4
3.5	Training and data stubs (honest placeholders)	4
4	Architecture diagrams	4
5	Verification: proving “done” and proving “not done”	4
6	Limitations and path to the full MVP	5
7	Appendix: mentor questions and build notes	6

1 Purpose of this prototype

The Meeting 1 goal, as stated in the brief, is to **validate architecture, confirm module division, and gather suggestions** — not to show a learning network. Concretely this report must convince a mentor of three things:

1. The team understands what an ANN is mathematically ($z = w \cdot x + b$, $output = activation(z)$) and can map each concept to a class or interface.
2. The proposed composition hierarchy and separation of training from the model are sound Object-Oriented Design.
3. Work can proceed in parallel without interface drift, because headers are frozen and unfinished parts fail loudly instead of silently.

Hence this prototype is **intentionally incomplete**: it compiles, its activation tests pass, its forward demo runs, and its training stub throws. An XOR network at this stage outputs essentially random values (≈ 0.43 for every input) — that *is* the expected result, because backpropagation does not exist yet.

1.1 Why clarity over speed

Production frameworks (PyTorch, TensorFlow) and matrix libraries (Eigen, BLAS) are banned by the brief. Every multiplication is a visible loop over `std::vector<double>`. The pay-off is pedagogical: a reader can trace a prediction from `NeuralNetwork::predict` through `Layer::forward` to `Neuron::forward` to `IActivation::activate` with no hidden machinery. Performance, GPU support, and autograd are explicitly out of scope.

1.2 Relation to the full MVP

`MiniANN_Early/` is the starting point; `MiniANN_MVP/` is the destination. The MVP adds: `backward()/gradient` accumulation, Xavier/He initialization, SGD and Adam, `Trainer::fit` with mini-batches, RFC-4180 CSV loading, accuracy/confusion-matrix metrics, 17-digit model serialization, console logging, and 100% XOR/Iris demos. Nothing in the early headers contradicts the MVP — the MVP only *fills in* the stubs defined here.

2 What the prototype does (and does not do)

Table 1 is the contract. “DONE” means compiled, tested, and demoed. “STUB” means the header exists so members can compile against it, but every call throws `std::logic_error("Meeting 1: ... not implemented")`.

Table 1: Meeting 1 scope: done vs. planned.

Module (owner)	Status	Evidence
Common types (<code>types.hpp</code>)	DONE	<code>Vector</code> , <code>Matrix</code> aliases
Activations (<code>activation.hpp</code>)	DONE	<code>test_activation.exe</code> passes
Neuron forward (M2)	DONE (forward only)	<code>forward_demo.exe</code> runs
Layer / Network forward (M1)	DONE (forward only)	dim check throws <code>invalid_argument</code>
<code>MSELoss::compute</code> (M3)	PARTIAL	value only, no gradient yet
<code>SGD::step</code> , backprop, <code>Trainer</code> (M3)	STUB	throws, demo catches it
<code>Dataset::loadCSV</code> , accuracy (M4)	STUB	throws
Serialization, logging, CLI demos (M4)	NOT STARTED	absent by design

In particular the program **cannot learn**: there is no loss gradient, no weight update, no data loader, and no metric. The forward demo computes an MSE *value* purely to show the loss interface shape, then deliberately calls the optimizer stub to prove it throws.

3 How we did it: module by module

3.1 Common types

namespace `miniann` holds using `Vector = std::vector<double>` and using `Matrix = std::vector<std::vector<double>>`. Every later header includes only `types.hpp` plus what it needs, so physical dependencies mirror the logical diagram in Fig. 1. No raw `new/delete` anywhere; memory is owned by `vector` and `unique_ptr`.

3.2 Activation subsystem (the completed piece)

The brief’s interface is reproduced verbatim:

```
1 class IActivation {
2 public:
3     virtual ~IActivation() = default;
4     virtual double activate(double z) const = 0;
5     virtual double derivative(double z) const = 0;
6 };
7 class Sigmoid : public IActivation {};
8 class Tanh : public IActivation {};
9 class ReLU : public IActivation {};
```

Why an abstract base? Each neuron must hold *some* non-linearity without knowing which one at compile time. A `unique_ptr<IActivation>` member plus virtual dispatch gives exactly that: the same `Neuron::forward` line `activation->activate(z)` executes sigmoid, tanh, or ReLU depending on what the factory closure injected. The derivative is declared now — even though no caller uses it yet — because backpropagation will need $f'(z)$ evaluated at the *pre-activation* z , and freezing that signature in Meeting 1 prevents the classic *z-vs-a* integration bug later.

Mathematics implemented in `src/activation.cpp`:

$$\sigma(z) = \frac{1}{1 + e^{-z}}, \quad \sigma'(z) = \sigma(z)(1 - \sigma(z)), \quad \tanh'(z) = 1 - \tanh^2(z), \quad \text{ReLU}(z) = \max(0, z), \quad \text{ReLU}'(z) = 1 \quad (1)$$

Uniform initialization ($w \sim U(-1, 1)$, $b \sim U(-0.5, 0.5)$) is used for now; Xavier/He variance preservation is explicitly deferred to the training phase.

3.3 Neuron: the smallest unit

```
1 class Neuron {
2 public:
3     Neuron(std::size_t numInputs, std::unique_ptr<IActivation> act,
4           std::mt19937& rng);
5     double forward(const Vector& inputs); // z = w.x + b, return
        activation(z)
6     const Vector& weights() const { return weights_; }
7     double bias() const { return bias_; }
8 private:
9     Vector weights_; double bias_ = 0.0;
10    std::unique_ptr<IActivation> activation_;
11 };
```

Encapsulation is strict: weights and bias are private, readable but not writable from outside (no public data, no setters yet — setters arrive with the serializer in the MVP). There is no cached z , no δ , no gradient buffer, because nothing computes gradients yet. The constructor takes the RNG by reference so the network seed controls the whole experiment.

3.4 Layer and network: composition over inheritance

A `Layer` owns `std::vector<Neuron>` and maps forward over it; a `NeuralNetwork` owns `std::vector<Layer>` and chains them:

```

1 Vector Layer::forward(const Vector& inputs) {
2     Vector out; out.reserve(neurons_.size());
3     for (auto& n : neurons_) out.push_back(n.forward(inputs));
4     return out;
5 }
6 Vector NeuralNetwork::predict(const Vector& input) {
7     Vector out = input;
8     for (auto& l : layers_) out = l.forward(out);
9     return out;
10 }
```

`addLayer(numNeurons, numInputs, makeAct)` validates `numInputs == previous_layer_size` and throws `invalid_argument` otherwise — this is Member 1’s core contribution at this stage. A small factory helper (`useSigmoid()/useTanh()/useReLU()`) returning `std::function<std::unique_ptr<IAct>` lets each layer pick its own non-linearity without the network knowing concrete types. Crucially, `predict` is the *only* computation entry point; there is no `backward()`, no `zeroGradients()`, no training hook inside the model. That absence is the design statement: the network answers “what am I?”, the future trainer will answer “how do I learn?”.

3.5 Training and data stubs (honest placeholders)

`include/miniann/training.hpp` freezes the intended Member 3 interface (`ILoss`, `MSELoss::compute`, `IOptimizer`, `SGD`) while `src/training.cpp` implements only the pure value computation and makes `SGD::step` throw. Likewise `dataset.hpp` offers `add()/accessors` but `loadCSV` and the free `accuracy()` throw. This pattern — **compilable headers, throwing bodies** — lets all four members build against the same headers in parallel while making unfinished work impossible to mistake for working code (the forward demo catches the throw on purpose).

4 Architecture diagrams

The roadmap (Fig. 3 read bottom-up as time) is: 1. Design (this report), 2. Core Components (finish forward + dimensions), 3. Training (gradients, optimizer), 4. Integration and Testing (datasets, metrics, demos). Each phase adds code without changing frozen signatures.

5 Verification: proving “done” and proving “not done”

Build (no CMake required):

```

1 cd MiniANN_Early
2 .\build.bat
3 .\test_activation.exe
4 .\activation_demo.exe
5 .\forward_demo.exe
```

Expected output (abbreviated):

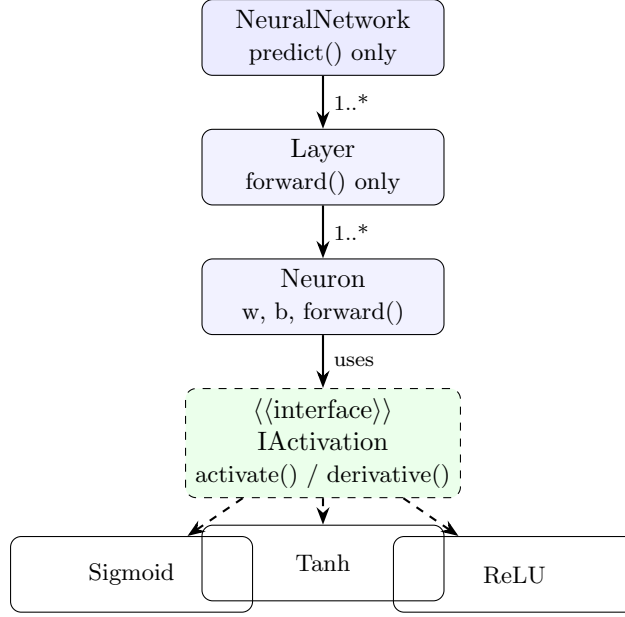


Figure 1: Composition hierarchy (done). The model owns forward prediction only; no training edge exists yet.

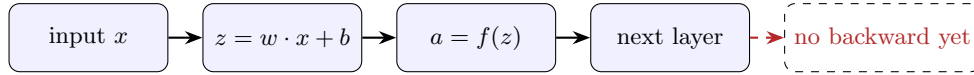


Figure 2: Forward data flow (solid = implemented; dashed red = explicitly absent in Meeting 1).

```

1 Build OK (Meeting 1: activation + forward only)
2 MEETING1 ACTIVATION TESTS PASS
3 MiniANN Meeting 1: activation module
4   z=0.0 -> 0.5000 (deriv 0.2500)
5   sigmoid(0)=0.5000 (expect 0.5)
6   relu(-2)=0.0000 (expect 0)
7 MiniANN Meeting 1: forward-only XOR (UNTRAINED)
8   [0,0] -> 0.4371 (mse-vs-0 0.1910)
9   [1,1] -> 0.4317 (mse-vs-0 0.1864)
10 training stub correctly throws: Meeting 1: SGD::step not implemented ...

```

`test_activation` asserts $\sigma(0) = 0.5$, $\sigma'(0) = 0.25$, $\tanh(0) = 0$, $\text{ReLU}(-2) = 0$, plus a virtual-dispatch check through `const IActivation*`. The forward demo builds a 2-2-1 tanh/sigmoid net (seed 42) and evaluates the four XOR corners *without training*: outputs cluster near 0.43 regardless of input, confirming the plumbing works and learning is absent. The caught `logic_error` is a positive test — it proves a reviewer cannot accidentally believe training exists.

6 Limitations and path to the full MVP

Everything below is out of scope for this snapshot and appears only as a stub or not at all: backpropagation and gradient accumulation, Xavier/He initialization, SGD/Adam stepping, mini-batch `Trainer::fit`, CSV ingestion and splits, accuracy/confusion-matrix metrics, `setprecision(17)` serialization with MINIANN 1 magic, logging, and any demo that learns (XOR at 100%, Iris). The upgrade path is additive: implement `Neuron::backward` ($\delta = \partial L / \partial a \cdot f'(z)$, accumulate ∇w), `Layer::backward` ($\partial L / \partial a_{\text{prev}} = W^T \delta$ with pre-step weights), `NeuralNetwork::backward`, the optimizer, and the data layer — without touching the activa-

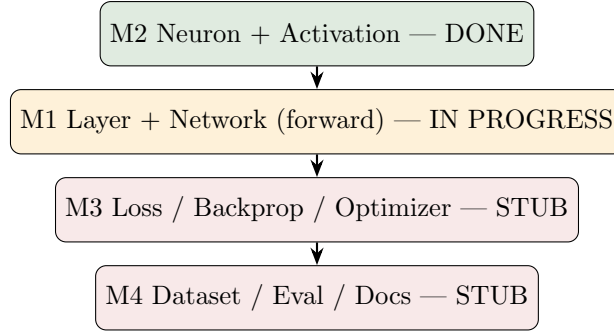


Figure 3: Module ownership and completion state. Color encodes status, arrows encode planned integration order.

tion headers frozen here.

7 Appendix: mentor questions and build notes

We ask: (1) is the training-outside-network split correct; (2) is the 4-member division balanced; (3) are activations, losses, optimizers, and metrics the right OOP seams? Build flags are `-std=c++17 -Wall -Wextra -Wpedantic`, zero raw `new/delete`, `const`-correct accessors, `PascalCase` classes / `I`-interfaces / `camelCase` methods / trailing-`_` members. Reproduce with `cmake -B build -S . && cmake --build build or build.bat`.